

Chapter 9: MLR Concepts and Mechanics

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to explain how multiple linear regression extends simple linear regression to estimate the conditional effect of each feature while holding others constant <{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to implement multiple linear regression in both Excel and Python (statsmodels) and interpret coefficient estimates, p-values, and R-squared <{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to dummy-code categorical variables and explain why one category must be dropped to avoid multicollinearity <{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to standardize numeric features to enable meaningful comparison of coefficient magnitudes across variables <{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to compute and interpret in-sample error metrics (MAE, RMSE) for regression models

9.1 Introduction

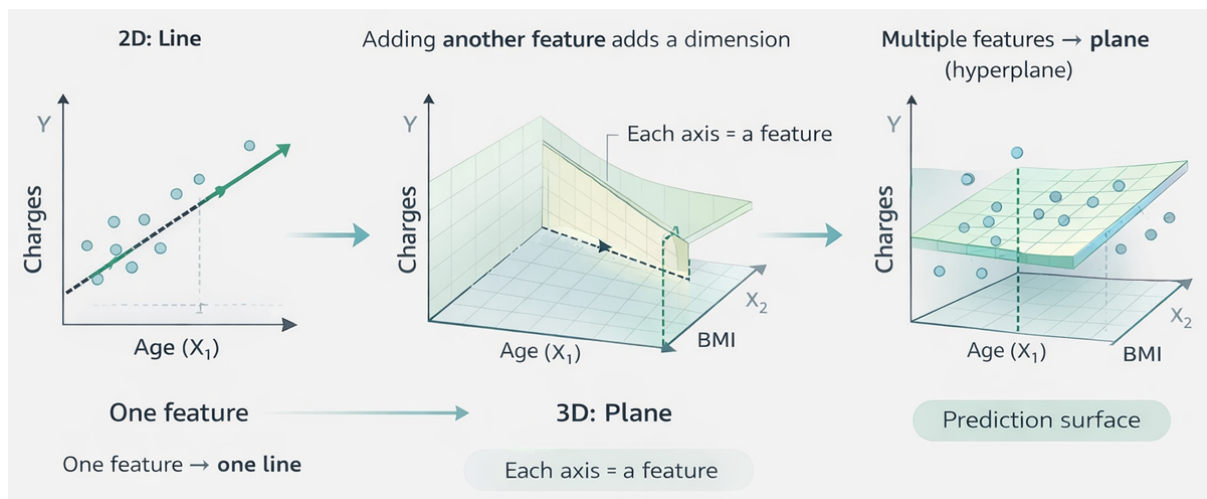


Figure 9.1: Multivariate versus Bivariate Analyses

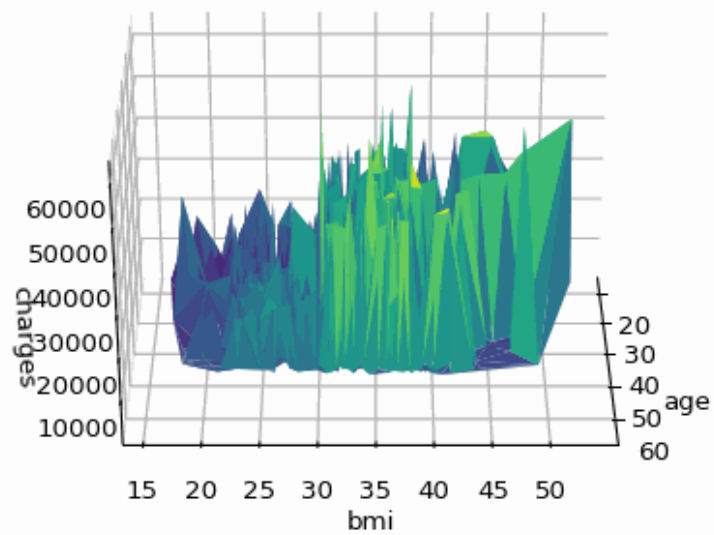


Figure 9.2: Animated 3D Trisurface Plot

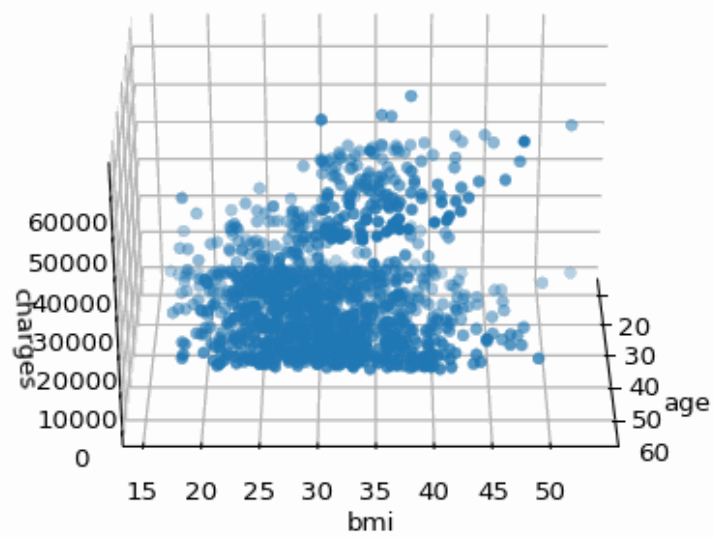


Figure 9.3: Animated 3D Scatterplot

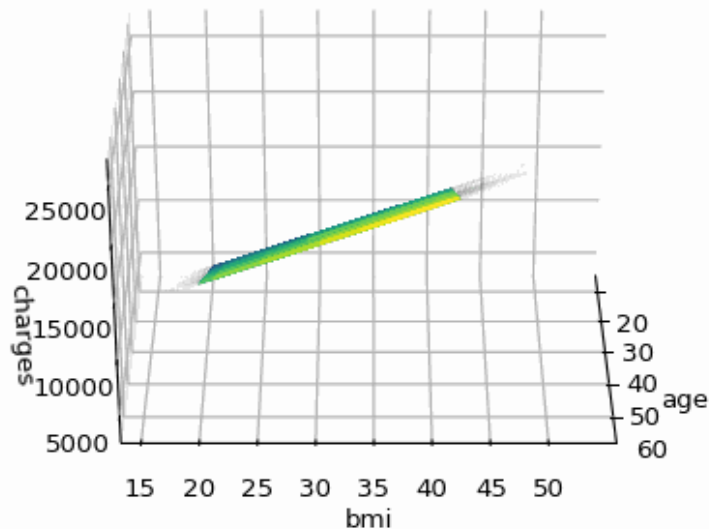


Figure 9.4: Animated 3D Plane Prediction

Now that you have learned how to collect, explore, and clean data, the next step in the data project process is to analyze that data in order to extract insights and generate predictions about outcomes of interest. This stage of the process is where modeling becomes central.

If you have taken a basic statistics course in high school or college (or even just algebra), you have likely worked with descriptive statistics and visualizations involving single variables (univariate analysis) and relationships between pairs of variables (bivariate analysis). While these approaches are useful for exploration and intuition, they are limited because they cannot account for the combined and interdependent effects of multiple features acting simultaneously.

Modeling The process of developing mathematical or computational functions that quantify the relationship between multiple input features and an outcome

of interest. Modeling involves creating functions that combine multiple features into a single equation. *Model training* refers to the process of using historical data to estimate the parameters (weights) that determine how strongly each feature contributes to the outcome. Once trained, a model can be applied to new data to explain relationships or to generate predictions for new observations.

Multiple linear regression is one of the most widely used modeling techniques because it serves two distinct but related purposes, depending on the analyst's goal:

- **Regression for causal (explanatory) inference:** The goal is to understand how changes in individual features are associated with changes in the outcome, holding other variables constant. In this setting, coefficient interpretation, statistical significance, and regression assumptions play a central role.
- **Regression for predictive inference:** The goal is to generate accurate predictions for new or unseen data. In this setting, predictive performance, generalization, and error metrics matter more than strict adherence to classical regression assumptions.

Although the same mathematical model underlies both uses, the questions we ask, the assumptions we emphasize, and the metrics we focus on differ substantially. This chapter begins by introducing the core concepts and mechanics of multiple linear regression. Subsequent chapters will then explore how regression is used differently for explanation versus prediction.

To illustrate the idea of multivariate modeling, consider the insurance dataset shown below. Earlier, you learned how to fit a regression line when predicting an outcome from a single feature. When predicting *charges* using

two features—*age* and *BMI*—the fitted model is no longer a line but a plane in three-dimensional space. Multiple linear regression generalizes this idea to many features, producing a multidimensional surface that best fits the data.

This chapter introduces one of the oldest, most interpretable, and still most widely used modeling techniques in analytics: *Multiple Linear Regression (MLR)*. Although modern data science increasingly relies on more complex machine learning models, linear regression remains foundational because it provides transparency, interpretability, and a clear link between data, assumptions, and decisions. Mastering MLR provides essential intuition that carries forward into more advanced modeling approaches.

9.2 Linear Regression

Background Theory

To understand multiple linear regression, it is useful to briefly review linear regression. You may recognize the familiar equation $y = mx + b$ from algebra courses, where m represents the slope of a line and b represents the y -intercept. In data analytics, linear regression is less about drawing a line and more about estimating how changes in one variable are associated with changes in an outcome, on average, using historical data.

A residual (i.e., error) is the difference between an actual value and the value predicted by the regression line. Each plotted point represents a single case (a row in a dataset) with values for one input feature and one outcome. For example, if y represents income and x represents age, each point reflects an individual's observed age and income. The fitted regression line—also called a line of best fit or trendline—produces predicted y values for given x values.

Linear regression chooses the line that minimizes the sum of squared residuals. Residuals are squared so that negative and positive errors do not cancel each other out and so that larger errors are penalized more heavily. Minimizing squared error also produces a unique, stable solution that can be computed efficiently, even for large datasets.

Model A formula—typically composed of a set of weights, a constant, and an error term—that estimates the expected value of an outcome given input features. A model uses historical data to estimate relationships between inputs and outcomes. These relationships are not guaranteed rules, but approximations that include uncertainty and error.

Suppose we train a simple linear regression model using age to predict income and obtain the following equation:

$$y = 6000x + 5$$

Review (if needed)

If you would like a refresher on how slope and intercept are calculated, optional review videos are available on Khan Academy, including [calculating slope and y-intercept](#) and [standard error](#).

These reviews are optional and not required if you already understand the general concept.

This equation represents an income prediction model trained on historical age and income data. Interpreting the equation:

- y is the predicted income.
- 6000 represents the estimated increase in income for each additional year of age.
- x is the observed age value.

- 5 is the predicted income when age equals zero, which illustrates a mathematical intercept rather than a realistic scenario.

This model can generate predictions for new data by substituting an age value into the equation. For example, the predicted income for a 40-year-old is:

$$\text{Predicted income} = 6000 \times 40 + 5$$

This results in a predicted income of \$240,005. The table below shows predictions for ages between 5 and 65.

$Y = 6000x + 5$							
Actual age	5	15	25	35	45	55	65
Predicted income	30005	90005	150005	210005	270005	330005	390005

Do you trust all of these predictions? While predictions for ages between roughly 45 and 65 may seem reasonable, a predicted income for a 5-year-old clearly is not. These implausible predictions do not mean regression is useless; instead, they indicate that model assumptions may not hold across all input values.

Evaluating assumptions such as linearity, constant variance, and appropriate data ranges is critical before relying on regression results. These ideas become even more important when extending from one predictor to many.

While the core principles of linear regression have remained stable for decades, modern analytics workflows increasingly rely on AI-assisted tools to estimate, validate, and interpret regression models efficiently and at scale.

With this refresher in mind, we now extend these ideas to multiple linear regression, where outcomes are predicted using several features

simultaneously.

9.3 Multiple Linear Regression

Multiple Linear Regression (MLR) A modeling technique that estimates the relationship between a single label and two or more features simultaneously. Multiple Linear Regression (MLR) extends simple linear regression by allowing many features to be used together to predict a single outcome. This is essential in real-world data, where outcomes are rarely driven by only one factor.

Note

This course focuses on understanding, applying, and interpreting MLR rather than deriving it mathematically. Students interested in the formal matrix-based derivation can consult the [Linear Regression Wikipedia page](#).

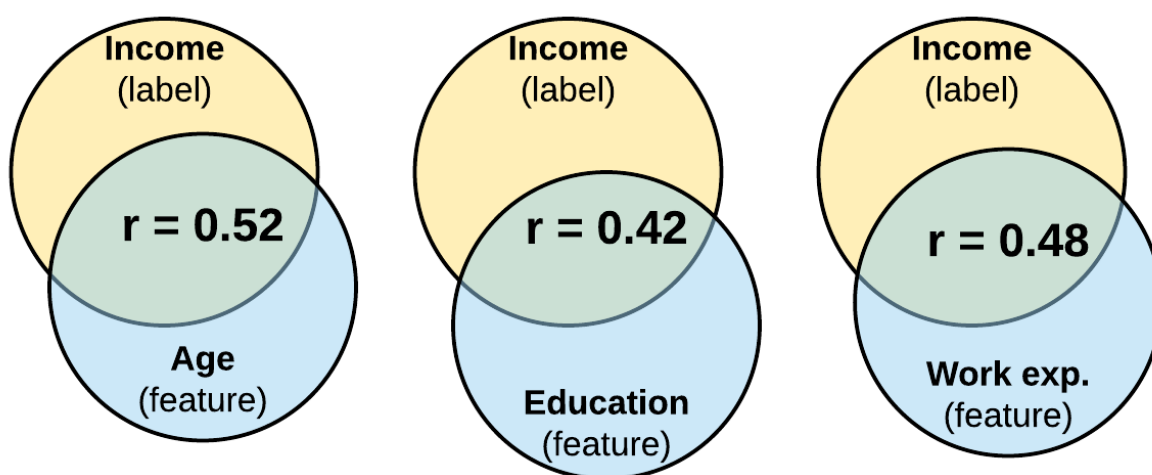
MLR produces one coefficient (β) for each feature and a single intercept term. This generalizes the familiar straight-line equation ($y = mx + b$) by replacing the single slope with multiple feature-specific weights. For example, a model predicting income (y) from age (x_1), education (x_2), and years of experience (x_3) takes the following form:

$$y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + b$$

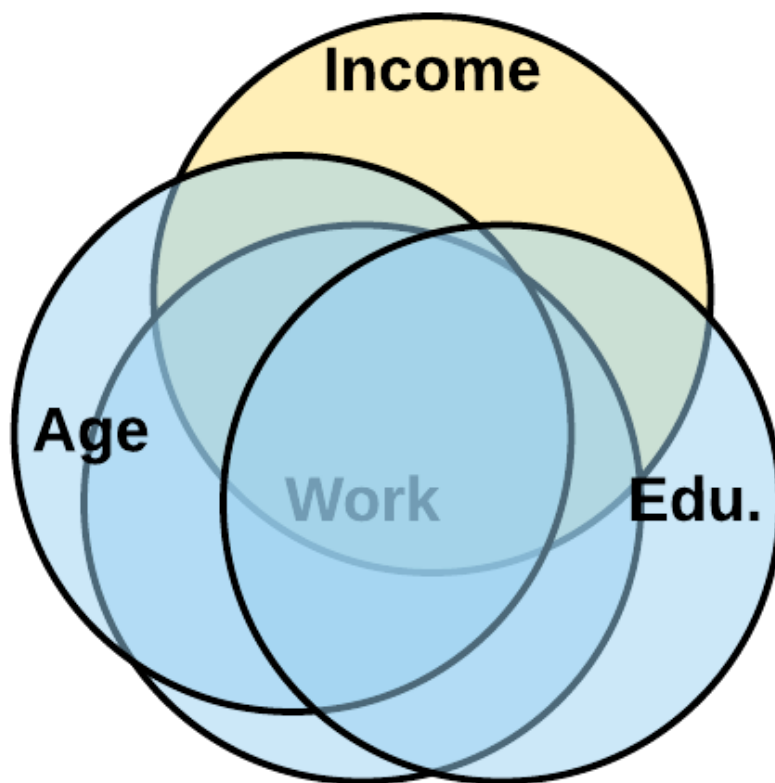
Each β coefficient represents the *conditional effect* of its feature—that is, how much the predicted label changes when that feature increases by one unit *while all other features are held constant*. This distinction is critical: unlike correlation, which examines variables in isolation, MLR isolates the unique contribution of each feature.

MLR estimates coefficients by finding the set of values that minimizes the total squared prediction error across all observations. Conceptually, this is the same least-squares principle used in simple linear regression, but applied in a higher-dimensional feature space. Because humans cannot visualize beyond three dimensions, this optimization is performed mathematically rather than graphically.

One of the main advantages of MLR is that it prevents misleading conclusions that arise from analyzing features one at a time. For example, suppose you compute bivariate correlations between income and three predictors: age ($r = 0.52$), education ($r = 0.42$), and work experience ($r = 0.48$). Examining these values alone might suggest that age is the most important predictor.



However, these features are themselves correlated with one another. Adding their correlations together ($0.52 + 0.48 + 0.42 = 1.42$) would imply more than perfect prediction, which is impossible. This occurs because shared information is being double- and triple-counted. Bivariate correlation cannot distinguish between unique and shared effects.



EFFECT SIZE

~~Age — $r = 0.52$~~

~~Education $r = 0.42$~~

~~Work exp. $r = 0.48$~~

$$r^2 = 0.61$$

MLR resolves this issue by estimating the *true* effect of each feature after accounting for overlap with other features. Individual effect sizes are captured by the β coefficients, while the overall explanatory power of the model is summarized by the coefficient of determination, R^2 .

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

R^2 measures the proportion of variability in the label explained by all features combined, relative to predicting the mean alone. Unlike correlation, it accounts for shared information among predictors and reflects the net explanatory power of the entire model.

Together, β coefficients and R^2 form the foundation for interpreting regression models. In the sections that follow, you will learn how to compute MLR models in Excel and Python, encode categorical features, scale inputs, test assumptions, and evaluate predictions—tasks that are increasingly automated using modern analytics tools and AI-assisted workflows.

9.4 MLR in Excel

Multiple Linear Regression can feel abstract for first-time modelers, and implementing it directly in Python introduces additional cognitive load due to syntax, libraries, and debugging. For many students, Excel provides a useful conceptual sandbox where model structure, feature estimates, model fit, multicollinearity, and prediction mechanics are easier to see and reason about before introducing code. If you are already comfortable with MLR concepts, you may skip this section. Otherwise, follow along with the videos below using the [insurance.csv](#) dataset.

This video can be viewed [online](#).

This video can be viewed [online](#).

This video can be viewed [online](#).

This video can be viewed [online](#).

This video can be viewed [online](#).

9.5MLR in Python

This video can be viewed [online](#).

MLR in Statsmodels

Now that you have built and interpreted a Multiple Linear Regression model in Excel, let's replicate that workflow in Python using the *statsmodels* package ([full documentation](#)). The goal here is not to introduce new concepts, but to show how the same model components—features, coefficients, intercepts, and model fit—appear when implemented programmatically.

As before, we divide the dataset into two parts: (1) the label (*y*) and (2) the set of numeric features (*X*) used to predict that label. We will again use the [insurance.csv](#) dataset.



```
import numpy as np
import pandas as pd
import statsmodels.api as sm

df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')

# Set label and features
y = df['charges']
X = df.select_dtypes(np.number).assign(const=1)
X = X.drop(columns=['charges'])
X.head()
```

	age	bmi	children	const
0	19	27.900	0	1
1	18	33.770	1	1
2	28	33.000	3	1
3	33	22.705	0	1
4	32	28.880	0	1

y is a Pandas Series because it contains a single column (the label), while X is a DataFrame containing multiple features. We intentionally restrict X to numeric columns because linear regression cannot operate directly on text data. The statement `.assign(const=1)` adds a column of ones that allows the model to estimate a y-intercept. In Statsmodels, omitting this column would force the regression plane through the origin, which is rarely appropriate.

Finally, we remove the label column from X . Including the label as a feature would trivially produce a perfect fit, which is both misleading and invalid.

Next, we fit the Multiple Linear Regression model using Ordinary Least Squares (OLS):



```
model = sm.OLS(y, X)
results = model.fit()

print(results.summary())
```

```
# Output:
#                                     OLS Regression Results
# =====
# Dep. Variable:                charges    R-squared:                0.120
# Model:                        OLS        Adj. R-squared:            0.118
# Method:                      Least Squares    F-statistic:                60.69
# Date:                        Wed, 12 Feb 2025    Prob (F-statistic):        8.80e-37
# Time:                        20:30:40        Log-Likelihood:            -14392.
# No. Observations:            1338        AIC:                      2.879e+04
# Df Residuals:                1334        BIC:                      2.881e+04
# Df Model:                    3
# Covariance Type:            nonrobust
# =====
#               coef    std err          t      P>|t|      [0.025    0.975]
# -----
# age            239.9945     22.289     10.767     0.000     196.269     283.720
# bmi            332.0834     51.310      6.472     0.000     231.425     432.741
# children       542.8647    258.241      2.102     0.036      36.261    1049.468
# const        -6916.2433    1757.480     -3.935     0.000    -1.04e+04    -3468.518
# =====
# Omnibus:                 325.395    Durbin-Watson:           2.012
# Prob(Omnibus):           0.000    Jarque-Bera (JB):        603.372
# Skew:                   1.520    Prob(JB):                9.54e-132
# Kurtosis:               4.255    Cond. No.                290.
# =====
#
# Notes:
# [1] Standard Errors assume that the covariance matrix of the errors is correctly
```

specified.

The regression summary contains a large amount of information. Rather than interpreting everything at once, we will focus on the most important sections and return to the others later in the chapter.

Descriptive Information

The upper-left portion of the output reports basic model metadata, including the dependent variable, estimation method (OLS), number of observations, and degrees of freedom for both the model and residuals. These values confirm that the model was constructed as intended.

Overall Model Quality (Model Fit)

In the upper-right portion, *R-squared* indicates the proportion of variance in insurance charges explained by the features in the model. In this case, $R^2 \approx 0.12$ means the model explains about 12% of the variation in charges. Whether this is “good” depends entirely on the context and on how well alternative models perform.

Adjusted R-squared A version of R-squared that accounts for the number of predictors in the model. Adjusted R-squared increases only when a new feature improves the model beyond what would be expected by chance. The similarity between R-squared and adjusted R-squared here indicates that each included feature contributes meaningfully.

Log-Likelihood (LL), *Akaike Information Criterion (AIC)*, and *Bayesian Information Criterion (BIC)* provide alternative ways to assess model quality while penalizing unnecessary complexity.

- *Log-Likelihood (LL)*: Higher values indicate a better fit.
- *AIC*: Balances goodness of fit with model complexity; lower is better.
- *BIC*: Similar to AIC but penalizes complexity more strongly.

At this stage, we primarily rely on R^2 to compare models predicting the same outcome. Feature selection strategies using AIC and BIC will be explored later.

Two additional performance measures not shown in the Statsmodels summary are **Mean Absolute Error (MAE)** The average absolute difference between predicted and actual values. and **Root Mean Squared Error (RMSE)** The square root of the average squared prediction error.. These metrics are computed using in-sample predictions meaning that we used the fitted model to make predictions for the same rows that were used to fit the model.



```
df_insample = pd.DataFrame({
    'Actual': df['charges'],
    'Predicted': model.fittedvalues,
    'Residuals': df['charges'] - model.fittedvalues
})

df_insample.head(10)
```

Output:

#	Actual	Predicted	Residuals (Error)
# 0	16884.92400	6908.777533	9976.146467
# 1	1725.55230	9160.977061	-7435.424761
# 2	4449.46200	12390.946918	-7941.484918
# 3	21984.47061	8543.527095	13440.943515
# 4	3866.85520	10354.147396	-6487.292196
# 5	3756.62160	9071.411158	-5314.789558
# 6	8240.58960	15771.234831	-7530.645231
# 7	7281.50560	12804.138689	-5522.633089
# 8	6406.41070	12955.328269	-6548.917569
# 9	28923.13692	16064.459249	12858.677671

MAE and RMSE quantify average prediction error, with RMSE placing greater emphasis on large errors. RMSE is therefore more sensitive to outliers and non-normal residuals.

It is important to understand a critical limitation of in-sample metrics like MAE and RMSE: they are calculated using predictions on the *same data* that was used to train the model.

In-sample metrics can be *optimistic*—meaning they may make the model appear better than it actually is. This happens because the model has already "seen" these data points during training. The model's coefficients were specifically chosen to minimize prediction error on this exact dataset, so it's not surprising that the model performs relatively well when evaluated on the same data.

Think of it like a student taking a practice exam and then immediately retaking the same exam: the second score will likely be higher because the student has already seen the questions and answers. Similarly, a model's in-sample performance may be artificially high because it has already "memorized" patterns specific to the training data.

The real question we want to answer is: *How well will this model perform on new, unseen data?* In-sample metrics cannot answer this question. A model might have excellent in-sample RMSE (low error on training data) but perform poorly on new data if it has overfitting. A modeling problem where the model learns patterns specific to the training data that don't generalize to new data, resulting in good training performance but poor test performance.—meaning it has learned patterns that are specific to the training data rather than generalizable relationships.

To assess how well a model will perform on new data, we need to evaluate it on *out-of-sample* data—data that was *not* used during model training. This requires splitting the data into separate training and test sets before building the model, then evaluating performance metrics (like MAE and RMSE) on

the test set. This process, along with strategies for preventing overfitting, will be covered in detail in a later chapter.

For now, recognize that in-sample metrics like MAE and RMSE are useful for understanding how well the model fits the training data, but they should not be used to make claims about how the model will perform on new data. Always interpret in-sample metrics with caution, understanding that they may be optimistic and that true predictive performance requires evaluation on unseen test data.

9.6 Feature Estimates

Coefficients (Feature Weights)

Once a multiple linear regression model has been fit, it produces a set of `coefficients ( $\beta$ )` Estimated weights that quantify the independent effect of each feature on the label after controlling for all other features in the model.. These coefficients appear in the lower half of the `OLS().summary()` output under the `coef` column.

Each coefficient represents the expected change in the label associated with a one-unit increase in the corresponding feature, holding all other features constant. This “holding constant” condition is critical: coefficients are not simple correlations, but *controlled effects* that isolate the portion of each feature’s influence that is not shared with other predictors.

At this stage, coefficients should be interpreted as *initial estimates*. They reflect what the model believes about feature effects given the current specification—but they are not yet guaranteed to be reliable. In later

sections, you will learn how diagnostic tests determine whether these estimates can be trusted.

Note: the overlaps are not drawn to scale; this is just for demonstration

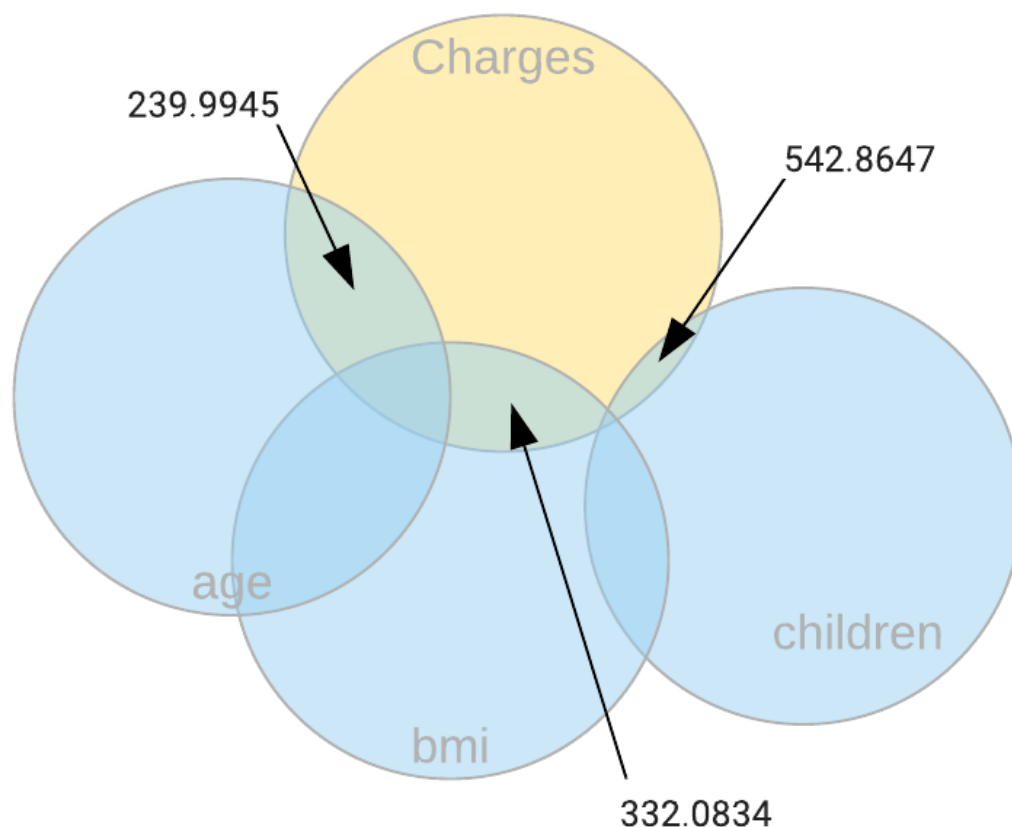


Figure 9.5: Conceptual Representation of MLR Coefficients

Once coefficients are estimated, predictions are generated using the general multiple linear regression equation:

$$y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + b$$

Based on the fitted model, the equation for predicting insurance charges takes the following form:

$$y = 239.99(\text{age}) + 332.08(\text{BMI}) + 542.86(\text{children}) - 6916.24$$

This equation can be used to compute predicted insurance charges for observations in the training data or for new customers. However, prediction accuracy and interpretability both depend on whether the model's assumptions are satisfied—an issue we will address later.

```
pd.set_option('display.max_columns', None)

df_formula = pd.DataFrame({
    'Prediction (y)': model.fittedvalues,
    '=': '=',
    'age β': round(model.params.iloc[0], 2),
    '*': '*',
    'age X': df['age'],
    '+': '+',
    'bmi β': round(model.params.iloc[1], 2),
    '*': '*',
    'bmi X': df['bmi'],
    '+': '+',
    'kids β': round(model.params.iloc[2], 2),
    '*': '*',
    'kids X': df['children'],
    '+': '+',
    'const': round(model.params.iloc[3], 2)
})

df_formula.head(5)
```

The trained model can also produce out-of-sample predictions Predictions for new cases that were not used during model training. using the *.predict()* method.

```
prediction = model.predict([32, 21, 2, 1])[0]
print(f"Predicted charges for age=32, bmi=21, children=2: ${round(prediction, 2)}")
```

The *const* term represents the y-intercept of the regression equation. It is estimated by including a column of ones when defining *X*. Without this column, the model would be forced through the origin—a restriction that is rarely justified in empirical data.

Interpretation Caution

At this point in the chapter, coefficients should be interpreted cautiously. Multicollinearity, non-linearity, autocorrelation, and heteroscedasticity can all distort coefficient estimates and their uncertainty. In Chapter 2, you will revisit these estimates after diagnosing and correcting assumption violations.

Standard Error

The *std err* column reports the standard error The estimated standard deviation of a coefficient's sampling distribution, reflecting the precision of the coefficient estimate.. Smaller standard errors indicate more stable estimates, while large standard errors often signal multicollinearity or insufficient information.

t-Statistics and p-Values

The *t*-statistic tests whether a coefficient is statistically distinguishable from zero and is computed as the coefficient divided by its standard error. Larger absolute *t*-values indicate stronger evidence that a feature contributes meaningfully to the model.

Raw coefficient magnitudes can be misleading when features are measured on different scales. Statistical significance (via *t*-statistics and *p*-values) helps contextualize coefficient importance—but these values are only meaningful if the underlying model assumptions hold.

The $P > |t|$ column reports *p*-values that quantify the probability of observing a coefficient as extreme as the one estimated if the true effect were zero. Smaller *p*-values suggest stronger evidence against the null hypothesis, but they should not be interpreted in isolation.

The $[0.025, 0.975]$ interval provides a 95% confidence interval for each coefficient. In Chapter 2, you will learn how diagnostics affect whether these intervals can be trusted.

9.7 Categorical Variables

This video can be viewed [online](#).

As you know, our MLR did not—and could not—use any categorical variables in the dataset. To include categorical variables, we must convert them into `dummy codes` Binary (0/1) variables that represent category membership for a categorical feature..

At this stage, our goal is simply to represent categorical information correctly in the model; we will not yet worry about whether the resulting coefficients are fully trustworthy.

We can modify our prior code to generate dummy variables in several ways using Pandas. Let's begin with the simplest technique: manually choosing the columns to dummy code.



```
import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')

# Manually enter column names to dummy code
df = pd.get_dummies(df, columns=['sex', 'smoker', 'region'])

df.head()
```

	age	bmi	children	charges	sex_female	sex_male	smoker_no	smoker_yes	region_northeast	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	True	False	False	True	False	False	False	True
1	18	33.770	1	1725.55230	False	True	True	False	False	False	True	False
2	28	33.000	3	4449.46200	False	True	True	False	False	False	True	False
3	33	22.705	0	21984.47061	False	True	True	False	False	True	False	False
4	32	28.880	0	3866.85520	False	True	True	False	False	True	False	False

Figure 9.6: Insurance Dataset with Dummy Codes

The first thing I want you to notice is that `.get_dummies()` may return values as True/False rather than 1/0. This has changed across Pandas versions because boolean values (True/False) can use less memory than integer values (1/0). The implication is that when we perform MLR, some packages (e.g., *statsmodels.api*) require dummy codes to be 0/1, while others (e.g., *sklearn*) will allow boolean True/False. You will see later that when using Statsmodels, we will cast True/False dummy variables to 0/1. But when we use sklearn to perform MLR, we will leave the dummy codes as True/False.

If we want more automated code that doesn't require manually entering column names, we can dynamically identify and encode all categorical columns instead of listing them:



```
df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')

# Use .select_dtypes to identify all columns that are categorical ('object' dtype)
dummies = df.select_dtypes(['object']).columns # Creates a list of categorical
column names
df = pd.get_dummies(df, columns=dummies) # Dummy code all categorical variables

df.head()
```

	age	bmi	children	charges	sex_female	sex_male	smoker_no	smoker_yes	region_northeast	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	True	False	False	True	False	False	False	True
1	18	33.770	1	1725.55230	False	True	True	False	False	False	True	False
2	28	33.000	3	4449.46200	False	True	True	False	False	False	True	False
3	33	22.705	0	21984.47061	False	True	True	False	False	True	False	False
4	32	28.880	0	3866.85520	False	True	True	False	False	True	False	False

Figure 9.7: Insurance Dataset with Dummy Codes

The table above shows the transformed dataset with dummy-coded categorical features. It is truncated for readability, but if you examine it in your notebook, you'll notice additional columns representing the different regions.

Now, let's test our MLR model with these new dummy-coded features included.



```
import statsmodels.api as sm

# Set label and features
y = df['charges']
X = df.drop(columns=['charges']).assign(const=1) # .assign(const=1) is a shorthand
method of creating a column of all 1s
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int) # Convert
True/False to 1/0
```

```
# Run the multiple linear regression model
model = sm.OLS(y, X).fit()
print(model.summary()) # View results
```

```
# Output:
#
# OLS Regression Results
# =====
# Dep. Variable:          charges    R-squared:                0.751
# Model:                  OLS       Adj. R-squared:           0.749
# Method:                 Least Squares   F-statistic:             500.8
# Date:                   Mon, 11 Sep 2023   Prob (F-statistic):      0.00
# Time:                   18:55:02    Log-Likelihood:         -13548.
# No. Observations:      1338        AIC:                    2.711e+04
# Df Residuals:          1329        BIC:                    2.716e+04
# Df Model:               8
# Covariance Type:       nonrobust
#
=====
#                               coef    std err          t      P>|t|      [0.025
0.975]
# -----
---
# age                256.8564      11.899      21.587      0.000      233.514
280.199
# bmi                339.1935      28.599      11.860      0.000      283.088
395.298
# children           475.5005     137.804       3.451      0.001      205.163
745.838
# sex_female         -82.5512     269.226      -0.307      0.759     -610.706
445.604
# sex_male          -213.8656     274.976      -0.778      0.437     -753.299
325.568
# smoker_no         -1.207e+04     282.338     -42.759      0.000     -1.26e+04
-1.15e+04
# smoker_yes         1.178e+04     313.530      37.560      0.000      1.12e+04
```

```

1.24e+04
# region_northeast    512.9050    300.348    1.708    0.088    -76.303
1102.113
# region_northwest    159.9411    301.334    0.531    0.596    -431.201
751.083
# region_southeast    -522.1170    330.759    -1.579    0.115    -1170.983
126.749
# region_southwest    -447.1459    310.933    -1.438    0.151    -1057.119
162.827
# const                -296.4168    430.507    -0.689    0.491    -1140.964
548.130
# =====
# Omnibus:                300.366    Durbin-Watson:                2.088
# Prob(Omnibus):          0.000    Jarque-Bera (JB):            718.887
# Skew:                   1.211    Prob(JB):                    7.86e-157
# Kurtosis:               5.651    Cond. No.                     7.13e+17
# =====

# Notes:
# [1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
# [2] The smallest eigenvalue is 6.91e-30. This might indicate that there are
# strong multicollinearity problems or that the design matrix is singular.

```

Take a look at the results. What do you see? Some aspects have improved, while others have worsened. First, R^2 has increased significantly from 12% to 75%, which is great. However, the assumption tests at the bottom remain essentially unchanged because we only added more features without transforming any of them. In fact, the condition number (*Cond. No.*) has increased dramatically to $7.13e+17$, signaling severe multicollinearity. Additionally, the notes at the bottom of the output warn that the design matrix may be singular, which is another symptom of redundancy in the feature set.

This occurs because we included redundant dummy variables. For example, *smoker_no* and *smoker_yes* provide the same information—if *smoker_yes* is 1, then *smoker_no* must be 0, and vice versa. Including both creates perfect multicollinearity. To resolve this, we remove one dummy-coded category (a reference group) for each categorical feature. In Pandas, we can do this by setting *drop_first=True* when generating dummy variables.



```
df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')

# Generate dummy variables for all categorical columns while dropping the first
category
df = pd.get_dummies(df, columns=df.select_dtypes(['object']).columns,
drop_first=True)

df.head()
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	False	True	False	False	True
1	18	33.770	1	1725.55230	True	False	False	True	False
2	28	33.000	3	4449.46200	True	False	False	True	False
3	33	22.705	0	21984.47061	True	False	True	False	False
4	32	28.880	0	3866.85520	True	False	True	False	False

Figure 9.8: Insurance Dataset with Dummy Codes After Dropping First

Now, let's rerun our model after removing the redundant dummy-coded categories.



```
# Set label and features
y = df['charges']
X = df.drop(columns=['charges']).assign(const=1)
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int)

# Generate model results
model = sm.OLS(y, X).fit()
print(model.summary())

# Output:
#
# OLS Regression Results
#
# Dep. Variable: charges R-squared: 0.751
# Model: OLS Adj. R-squared: 0.749
# Method: Least Squares F-statistic: 500.8
# Date: Wed, 12 Feb 2025 Prob (F-statistic): 0.00
# Time: 17:41:02 Log-Likelihood: -13548.
# No. Observations: 1338 AIC: 2.711e+04
# Df Residuals: 1329 BIC: 2.716e+04
# Df Model: 8
# Covariance Type: nonrobust
#
=====
# coef std err t P>|t| [0.025
0.975]
# -----
---
# age 256.8564 11.899 21.587 0.000 233.514
280.199
```

```

# bmi          339.1935    28.599    11.860    0.000    283.088
395.298
# children     475.5005    137.804     3.451    0.001    205.163
745.838
# sex_female   -82.5512    269.226    -0.307    0.759   -610.706
445.604
# sex_male     -213.8656    274.976    -0.778    0.437   -753.299
325.568
# smoker_no    -1.207e+04    282.338   -42.759    0.000   -1.26e+04
-1.15e+04
# smoker_yes    1.178e+04    313.530    37.560    0.000    1.12e+04
1.24e+04
# region_northeast  512.9050    300.348     1.708    0.088    -76.303
1102.113
# region_northwest  159.9411    301.334     0.531    0.596   -431.201
751.083
# region_southeast -522.1170    330.759    -1.579    0.115  -1170.983
126.749
# region_southwest -447.1459    310.933    -1.438    0.151  -1057.119
162.827
# const        -296.4168    430.507    -0.689    0.491  -1140.964
548.130
# =====
# Omnibus:          300.366   Durbin-Watson:          2.088
# Prob(Omnibus):    0.000   Jarque-Bera (JB):       718.887
# Skew:             1.211   Prob(JB):               7.86e-157
# Kurtosis:         5.651   Cond. No.               7.13e+17
# =====
#
# Notes:
# [1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
# [2] The smallest eigenvalue is 6.91e-30. This might indicate that there are
# strong multicollinearity problems or that the design matrix is singular.

```

What changed? We now have fewer features in the model because we removed one category per categorical feature (the reference category). The R^2 value remains the same, but the condition number (*Cond. No.*) drops dramatically (from about $7.13e+17$ to roughly *311* in this example), which indicates the redundancy problem has been reduced. Interpreting dummy-coded coefficients also becomes more meaningful because each coefficient is now measured relative to its reference group.

Finally, let's check whether our *MAE* and *RMSE* values have improved.



```

# Calculate Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE)
print(f"MAE:\t${abs(model.fittedvalues - y).mean():.2f}&quot;")
print(f"RMSE:\t${((model.fittedvalues - y)**2).mean() ** (1/2):.2f}&quot;")

```

```
# Output
# MAE:    $4170.89
# RMSE:   $6041.68
```

This model predicts much better than the earlier numeric-only model. Our *MAE* decreases from 9000+ to about 4170, and our *RMSE* drops to around 6041, indicating a substantial improvement in fit.

In the next chapter, we will use diagnostics like VIF and residual analysis to further refine which categorical features we should trust and which should be removed.

9.8 Feature Scaling

Feature scaling adjusts the numeric range of features so they can be meaningfully compared within a regression model. While ordinary least squares (OLS) multiple linear regression can be estimated without scaling, scaling plays an important role when interpreting coefficients and comparing the relative strength of predictors.

In this chapter, feature scaling is introduced as a *descriptive and interpretive tool*, not as a corrective step. Scaling does not fix violated regression assumptions such as non-linearity, heteroscedasticity, or multicollinearity. Those issues are addressed explicitly in the next chapter using regression diagnostics.

Here, scaling helps answer a limited but important question: when predictors are measured in different units, how can coefficient magnitudes be compared in a principled way?

Terminology

Feature scaling A transformation that adjusts the numeric range of feature values so they are comparable across predictors. Scaling changes units of measurement but preserves the underlying relationships between variables.

In practice, two related terms are commonly used:

- **Normalization** A general term for rescaling features to a common numeric range. The most common example is min–max normalization, which rescales values to fall between 0 and 1.
- **Standardization** A specific form of scaling that transforms values into z-scores. Standardized features have a mean of zero and a standard deviation of one.

Although these terms are sometimes used interchangeably in casual discussion, they are mathematically distinct and serve different interpretive purposes.

It is critical to distinguish between *numeric predictors* and *dummy-coded categorical predictors* when scaling features. Only continuous numeric variables should be scaled.

Dummy-coded variables (0/1 indicators) should *not* be scaled. A value of 0 or 1 already represents a complete and meaningful unit change: membership versus non-membership in a category. Scaling dummy variables would distort their interpretation without providing any analytical benefit.

Why dummy variables are not scaled

Scaling a 0/1 variable would replace clear category membership with fractional values that have no real-world meaning. In regression models, dummy coefficients are interpreted relative to a reference group, and

preserving the 0/1 structure ensures that interpretation remains valid and intuitive.

Accordingly, when scaling is applied in this chapter, it is applied only to continuous numeric predictors. Dummy-coded categorical variables are left unchanged.

Scaling in Python

Python's scikit-learn library provides several common scaling utilities. In this section, we focus on two widely used approaches: *StandardScaler* and *MinMaxScaler*. Both preserve the underlying relationships in the data while changing the units on which numeric predictors are measured.

To illustrate the effects of scaling, we return to the insurance dataset. Categorical variables are dummy-coded, and only continuous numeric predictors are scaled. Dummy variables are intentionally excluded from scaling.



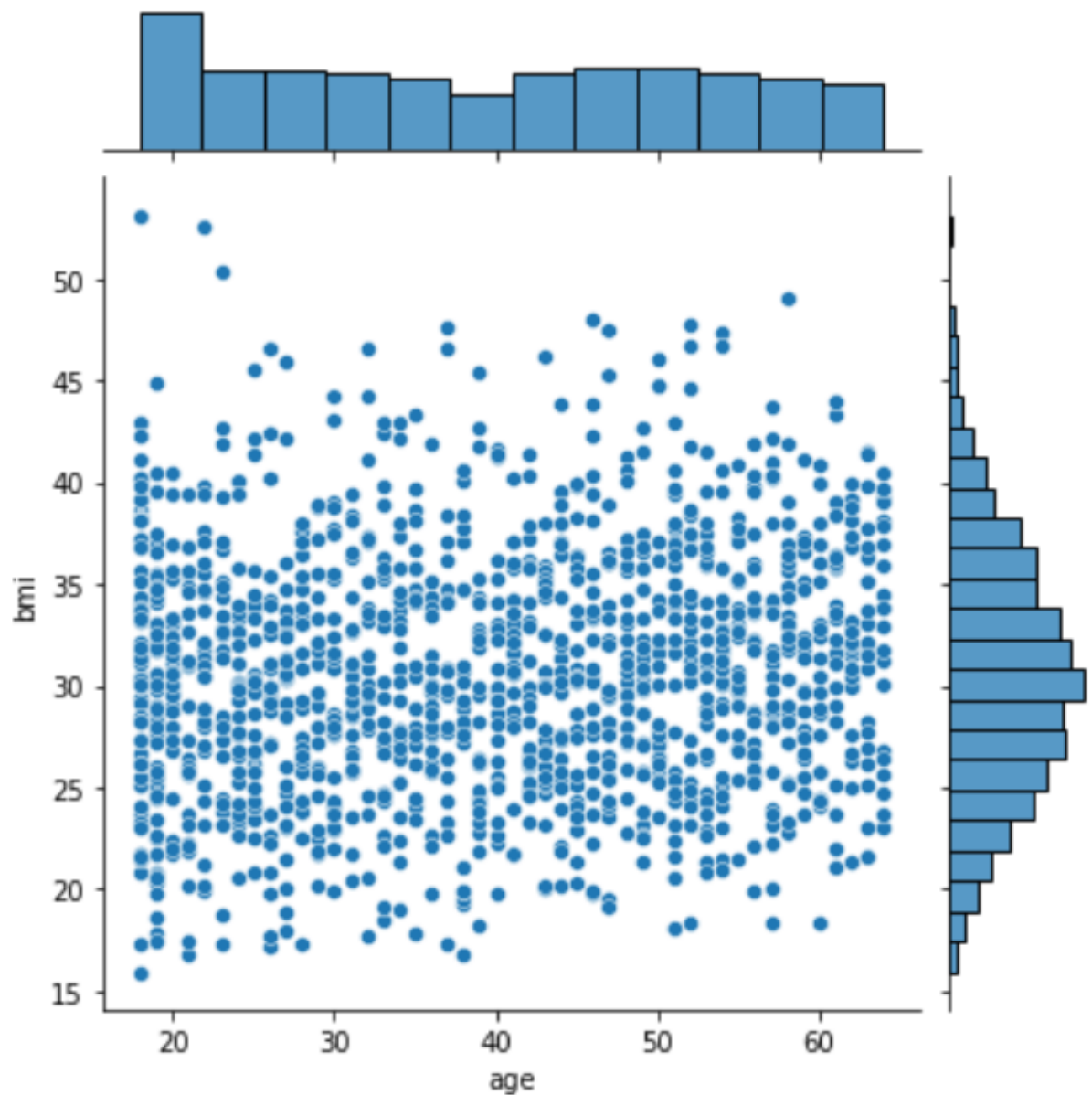
```
from sklearn import preprocessing
import pandas as pd
import seaborn as sns

df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/data/insurance.csv')

# Dummy-code categorical variables
df = pd.get_dummies(df, columns=df.select_dtypes(['object']).columns,
drop_first=True)

# Identify numeric predictors only
numeric_cols = ['age', 'bmi', 'children']

sns.jointplot(df, x='age', y='bmi');
```



Standardization (Z-Score Scaling)

Standardization converts numeric predictors into z-scores by subtracting the mean and dividing by the standard deviation. After standardization, coefficients represent the expected change in the label associated with a one-standard deviation increase in a predictor, holding all other variables constant.



```
df_zscore = df.copy()
df_zscore[numeric_cols] =
preprocessing.StandardScaler().fit_transform(df_zscore[numeric_cols])
df_zscore.head()
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	-1.438764	-0.453320	-0.908614	0.298584	-1.010519	1.970587	-0.566418	-0.611324	1.765481
1	-1.509965	0.509621	-0.078767	-0.953689	0.989591	-0.507463	-0.566418	1.635795	-0.566418
2	-0.797954	0.383307	1.580926	-0.728675	0.989591	-0.507463	-0.566418	1.635795	-0.566418
3	-0.441948	-1.305531	-0.908614	0.719843	0.989591	-0.507463	1.765481	-0.611324	-0.566418
4	-0.513149	-0.292556	-0.908614	-0.776802	0.989591	-0.507463	1.765481	-0.611324	-0.566418

Notice above that we didn't scale the label charges. In causal regression modeling, feature predictors may be scaled for interpretability, but the label should remain in its original units. Scaling the label is reserved for prediction-focused workflows. Otherwise, as you'll notice if you scale the label, you won't get feature coefficients in a meaningful scale.

Min–Max Normalization

Min–max normalization rescales numeric predictors to a fixed range between 0 and 1. Coefficients then represent the expected change in the label associated with moving from the minimum to the maximum observed value of a predictor.



```
df_minmax = df.copy()
df_minmax[numeric_cols] =
preprocessing.MinMaxScaler().fit_transform(df_minmax[numeric_cols])
df_minmax.head()
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	0.021739	0.321227	0.0	0.251611	0.0	1.0	0.0	0.0	1.0
1	0.000000	0.479150	0.2	0.009636	1.0	0.0	0.0	1.0	0.0
2	0.217391	0.458434	0.6	0.053115	1.0	0.0	0.0	1.0	0.0
3	0.326087	0.181464	0.0	0.333010	1.0	0.0	1.0	0.0	0.0
4	0.304348	0.347592	0.0	0.043816	1.0	0.0	1.0	0.0	0.0

Effect on Regression Interpretation

Scaling does not change model fit statistics such as R^2 , F-statistics, or residual patterns. These quantities depend on the underlying relationships in the data, not on the units of measurement.

What scaling changes is coefficient magnitude and interpretability. After scaling numeric predictors, coefficients can be compared meaningfully across predictors, while their signs and statistical significance remain unchanged.

Notice if we run our MLR again with the MinMax scaled numeric features, we can meaningfully compare them to each other even though they were previously on different scales (because now they're on the same scale).



```
import statsmodels.api as sm

y = df_minmax.charges
X = df_minmax.drop(columns=['charges']).assign(const=1)

# Convert boolean columns to integers (0s and 1s)
X[X.select_dtypes(bool).columns] = X.select_dtypes(bool).astype(int)

model = sm.OLS(y, X).fit()
print(model.summary())
```

```
# Output:
#
# OLS Regression Results
# =====
# Dep. Variable:          charges    R-squared:                0.751
# Model:                  OLS        Adj. R-squared:           0.749
# Method:                 Least Squares    F-statistic:             500.8
# Date:                   Tue, 06 Jan 2026    Prob (F-statistic):       0.00
```

```

# Time: 20:14:42 Log-Likelihood: -13548.
# No. Observations: 1338 AIC: 2.711e+04
# Df Residuals: 1329 BIC: 2.716e+04
# Df Model: 8
# Covariance Type: nonrobust
#
=====
# coef std err t P>|t| [0.025
0.975]
# -----
--
# age 1.182e+04 547.347 21.587 0.000 1.07e+04
1.29e+04
# bmi 1.261e+04 1063.042 11.860 0.000 1.05e+04
1.47e+04
# children 2377.5027 689.020 3.451 0.001 1025.816
3729.189
# sex_male -131.3144 332.945 -0.394 0.693 -784.470
521.842
# smoker_yes 2.385e+04 413.153 57.723 0.000 2.3e+04
2.47e+04
# region_northwest -352.9639 476.276 -0.741 0.459 -1287.298
581.370
# region_southeast -1035.0220 478.692 -2.162 0.031 -1974.097
-95.947
# region_southwest -960.0510 477.933 -2.009 0.045 -1897.636
-22.466
# const -1901.5967 586.973 -3.240 0.001 -3053.091
-750.103
# =====
# Omnibus: 300.366 Durbin-Watson: 2.088
# Prob(Omnibus): 0.000 Jarque-Bera (JB): 718.887
# Skew: 1.211 Prob(JB): 7.86e-157
# Kurtosis: 5.651 Cond. No. 9.59
# =====
#
# Notes:
# [1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.

```

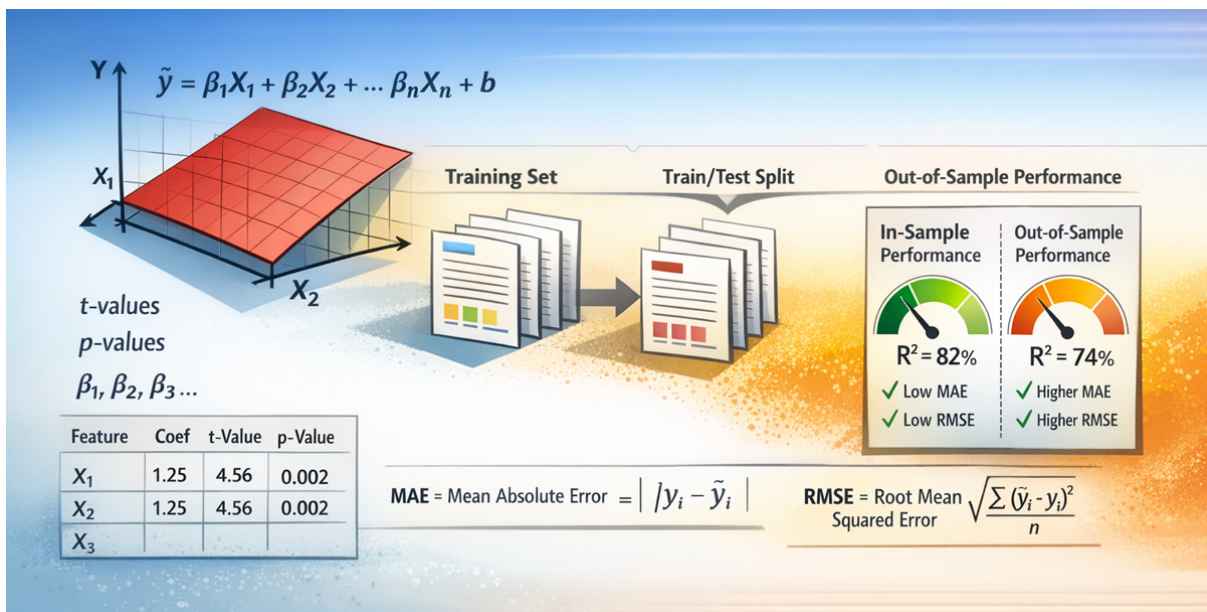
After scaling the numeric predictors, the coefficients for *bmi* and *children* are now expressed on a comparable scale. This allows us to compare their relative effect sizes more meaningfully. In this scaled model, *bmi* exhibits a larger coefficient magnitude than *children*, suggesting a stronger association with insurance charges, holding all other variables constant. In the unscaled model, the larger coefficient for *children* reflected differences in measurement units rather than a stronger underlying relationship.

More generally, scaling alters measurement units but does not change the underlying relationships in the data. For this reason, scaling should be viewed as an interpretive aid rather than a corrective technique. Coefficients

examined here remain provisional and should be revisited after diagnostic assumptions are evaluated in the next chapter.

In the next chapter, feature estimates are revisited after evaluating regression assumptions. At that stage, scaled coefficients become more trustworthy inputs for interpretation and feature selection.

9.9 Summary



In this chapter, you were introduced to multiple linear regression (MLR) as a foundational modeling technique for understanding relationships between features and a continuous outcome. The emphasis throughout was on building, estimating, and interpreting regression models in a controlled, explanatory setting.

What MLR Provides

MLR allows us to estimate the independent association between each feature and the label while controlling for the presence of other features in the model. This makes MLR especially valuable for *causal reasoning and decision support*, where understanding direction, magnitude, and relative importance of predictors matters.

Using Python and the Statsmodels OLS implementation, you learned how to define features and labels, fit a regression model, and interpret the resulting output, including coefficients, standard errors, t-statistics, and p-values.

Feature Estimates as a Starting Point

Regression coefficients provide an initial estimate of how changes in each feature are associated with changes in the outcome, holding all other features constant. These estimates form the basis for interpretation, comparison, and reasoning about potential drivers of outcomes.

At this stage, however, feature estimates should be treated as *provisional*. While they are mathematically correct given the fitted model, their reliability depends on assumptions that have not yet been fully evaluated or corrected.

Preparing Features for Modeling

Because regression models require numeric inputs, categorical variables were converted into dummy-coded features. You also saw why one category must be removed for each categorical feature to avoid redundancy and perfect multicollinearity.

You also explored feature scaling, including standardization and min–max normalization. While MLR does not require scaling to function, scaling

improves coefficient comparability and prepares the data for algorithms that are sensitive to feature magnitude.

A Deliberate Pause Before Trust

Although you examined model fit statistics and feature estimates, this chapter intentionally stopped short of validating regression assumptions or refining the model. This pause is deliberate.

Before coefficients can be trusted for strong claims or downstream decisions, the model must be evaluated for normality, multicollinearity, autocorrelation, linearity, and homoscedasticity. These diagnostics—and the adjustments they motivate—are addressed in the next chapter.

Where This Leads

By the end of this chapter, you should be comfortable building and interpreting an initial regression model, understanding what the output tells you, and recognizing its limitations.

In the next chapter, you will revisit regression models with a more critical lens, applying diagnostics, transformations, and refinements that allow feature estimates to move from *suggestive* to *defensible*, and ultimately preparing the model for predictive use.

9.10 Case Studies

This section includes three practice assessments using new datasets so you can rehearse the same multiple linear regression workflow with different feature mixes and real-world context.

For each dataset, your goal is to build an MLR model, interpret fit statistics and coefficients, and answer a set of targeted analysis questions.

Case #1: Diamonds Dataset

This practice uses the **Diamonds** dataset that ships with the Seaborn Python package. Seaborn makes the dataset available through a built-in loader function, so you can download it directly in your notebook without any external files.

Dataset attribution: The Diamonds dataset is distributed with the Seaborn data repository and can be loaded with `seaborn.load_dataset("diamonds")`. If you want the underlying CSV source, Seaborn hosts it in its public GitHub repository under `seaborn-data`. To be clear, you can get this dataset using this code:

```
import seaborn as sns

# Download/load the dataset from Seaborn
df = sns.load_dataset("diamonds")
```

In this chapter, you are practicing **MLR for causal (explanatory) modeling**. That means you will:

- Load the dataset using `seaborn.load_dataset("diamonds")`.
- Inspect the dataset: rows/columns, data types, and summary statistics.
- Fit an MLR model predicting *price* using numeric predictors (*carat*, *depth*, *table*, *x*, *y*, *z*) and categorical predictors (*cut*, *color*, *clarity*).
- Record model fit metrics (R^2 and *Adjusted R^2*).
- Identify the most statistically significant predictors using *t* and $P > |t|$.
- (Optional) Compute standardized coefficients for numeric predictors and compare magnitudes.

Analytical questions (answers should be specific)

1. How many rows and columns are in the Diamonds dataset?
2. What is the mean value of *price* in the dataset?
3. What are the R^2 and *Adjusted R^2* values for your fitted MLR model?
Report both values to 4 decimal places.
4. Which single predictor term (feature or dummy-coded category) has the smallest $P > |t|$ value in your model output?
5. Among the numeric predictors (*carat*, *depth*, *table*, *x*, *y*, *z*), which one has the largest absolute *t*-value? Provide the feature name and its *t*-value (rounded to 2 decimals).
6. Pick one categorical variable (*cut*, *color*, or *clarity*). Which category level appears to increase predicted *price* the most relative to the reference group (based on coefficient sign and magnitude)? Provide the dummy term name and coefficient value.
7. (Optional) After standardizing numeric variables, which numeric predictor has the largest standardized coefficient magnitude? Provide the feature name and the standardized coefficient value.



Diamonds Practice Answers

These answers were computed by fitting an OLS multiple linear regression model predicting *price* using numeric predictors (*carat*, *depth*, *table*, *x*, *y*, *z*) plus dummy-coded categorical predictors for *cut*, *color*, and *clarity* (with the first category as the reference group for each categorical variable).

1. The Diamonds dataset contains 53940 rows and 10 columns.

2. The mean value of *price* is 3932.7997.
3. For the fitted MLR model, $R^2 = 0.9198$ and *Adjusted R*² = 0.9197 (both reported to 4 decimals).
4. The single predictor term with the smallest $P > |t|$ value is *carat* (its p-value is effectively 0.0000 in floating-point terms and is shown as 0.000 in typical summary output due to rounding).
5. Among numeric predictors, *carat* has the largest absolute *t*-value: $t = 231.49$ (rounded to 2 decimals).
6. Using *cut* as the categorical variable, the level that increases predicted *price* the most relative to the reference group is *cut_Ideal* with coefficient 832.91.
7. (Optional) After standardizing numeric variables, the numeric predictor with the largest standardized coefficient magnitude is *carat* with standardized coefficient 5335.88.

Case #2: Red Wine Quality Dataset

This practice uses the **Red Wine Quality** dataset (physicochemical measurements plus a quality rating). Unlike the Diamonds dataset, this dataset contains only numeric predictors, so you will practice building and interpreting an MLR model without dummy coding.

Dataset attribution: This dataset is commonly distributed as the *winequality-red.csv* file from the UCI Machine Learning Repository (Wine Quality Data Set), originally published by Cortez et al. in “Modeling wine preferences by data mining from physicochemical properties” (Decision Support Systems, 2009). In this course, you may be given the CSV directly (as *winequality-red.csv*), or you can download it from UCI and load it into your notebook.

This download can be found [online](#).

To load the dataset, use one of the following approaches (choose one):

- **Option A (recommended if the CSV is provided):** Upload *winequality-red.csv* to your notebook environment, then load it with `pd.read_csv()`.
- **Option B (download from UCI):** Download the CSV from the UCI repository, then load it with `pd.read_csv()`.

In this chapter, you are practicing **MLR for causal (explanatory) modeling**. That means you will:

- Load the dataset and inspect it: rows/columns, data types, and summary statistics.
- Fit an OLS MLR model predicting *quality* using all other columns as numeric predictors.
- Record model fit metrics (R^2 and *Adjusted R^2*).
- Identify the most statistically significant predictors using t and $P > |t|$.
- (Optional) Compute standardized coefficients for numeric predictors and compare magnitudes.

Analytical questions (answers should be specific)

1. How many rows and columns are in the Red Wine Quality dataset?
2. What is the mean value of *quality* in the dataset?
3. What are the R^2 and *Adjusted R^2* values for your fitted MLR model?
Report both values to 4 decimal places.
4. Which single predictor term has the smallest $P > |t|$ value in your model output?

5. Which predictor has the largest absolute t -value? Provide the predictor name and its t -value (rounded to 2 decimals).
6. Which predictor has the largest positive coefficient in the fitted model? Provide the predictor name and coefficient value (rounded to 4 decimals).
7. (Optional) After standardizing numeric predictors (z-scores) and refitting the model, which predictor has the largest standardized coefficient magnitude? Provide the predictor name and coefficient value (rounded to 4 decimals).



Red Wine Quality Practice Answers

These answers were computed by fitting an OLS multiple linear regression model predicting *quality* using all remaining columns in *winequality-red.csv* as numeric predictors, with an intercept term (*const*).

1. The Red Wine Quality dataset contains 1599 rows and 12 columns.
2. The mean value of *quality* is 5.6360.
3. For the fitted MLR model, $R^2 = 0.3606$ and *Adjusted R^2* = 0.3561 (both reported to 4 decimals).
4. The single predictor term with the smallest $P > |t|$ value is *alcohol* (its p-value is effectively 0.0000 in floating-point terms and is shown as 0.000 in typical summary output due to rounding).
5. The predictor with the largest absolute t -value is *alcohol* with $t = 10.43$ (rounded to 2 decimals).
6. The predictor with the largest positive coefficient is *sulphates* with coefficient 0.9163 (rounded to 4 decimals).

7. (Optional) After standardizing numeric predictors (z-scores) and refitting the model (without scaling the label), the predictor with the largest standardized coefficient magnitude is *alcohol* with standardized coefficient 0.2942 (rounded to 4 decimals).

Case #3: Bike Sharing Daily Dataset

This practice uses the **Bike Sharing** daily dataset (the *day.csv* file). You will fit an OLS multiple linear regression model to explain variation in total daily rentals (*cnt*) using both numeric predictors (weather conditions) and categorical predictors (season/month/weekday/weather situation).

Dataset attribution: This dataset is distributed as part of the Bike Sharing Dataset hosted by the UCI Machine Learning Repository (Fanaee-T and Gama). It includes daily rental counts and weather/context variables derived from the Capital Bikeshare system in Washington, D.C. You will use the *day.csv* file provided with your course materials.

This download can be found [online](#).

In this chapter, you are practicing **MLR for causal (explanatory) modeling**. That means you will fit the model on the full dataset (no train/test split) and interpret model fit and feature evidence using R^2 , *Adjusted R^2* , *t*-values, and $P > |t|$.

Important modeling note: Do not include *casual* or *registered* as predictors because they directly sum to *cnt* and would leak the answer into the model.

Tasks

- Inspect the dataset: rows/columns, data types, and summary statistics for *cnt*.

- Fit an OLS MLR model predicting *cnt* using numeric predictors (*temp*, *atemp*, *hum*, *windspeed*) and categorical predictors (*season*, *mnth*, *weekday*, *weathersit*), plus binary indicators (*yr*, *holiday*, *workingday*).
- Dummy-code the categorical predictors using *drop_first=True*, then fit the model with Statsmodels *OLS*.
- Record model fit metrics (R^2 and *Adjusted R^2*).
- Identify the most statistically significant predictor term using $P > |t|$.
- Compute standardized coefficients for the numeric predictors (*yr*, *holiday*, *workingday*, *temp*, *atemp*, *hum*, *windspeed*) while leaving dummy codes unscaled, then compare effect sizes.

If you want a code scaffold that matches the chapter style (no formula interface), start here:

```
import pandas as pd
import statsmodels.api as sm

df = pd.read_csv('day.csv')

# Label
y = df['cnt']

# Predictors (exclude leakage variables such as casual and registered)
X = df[['season', 'yr', 'mnth', 'holiday', 'weekday', 'workingday', 'weathersit', 'temp', 'atemp', 'hum', 'windspeed']]

# Dummy-code categorical predictors (reference group is the first category for each)
X = pd.get_dummies(X, columns=['season', 'mnth', 'weekday', 'weathersit'], drop_first=True)

# Statsmodels expects numeric (0/1) dummies, not True/False
bool_cols = X.select_dtypes('bool').columns
X[bool_cols] = X[bool_cols].astype(int)

# Add intercept
X = X.assign(const=1)

model = sm.OLS(y, X).fit()
print(model.summary())
```

Analytical questions (answers should be specific)

1. How many rows and columns are in the Bike Sharing daily dataset (*day.csv*)?
2. What is the mean value of *cnt* in the dataset?
3. What are the R^2 and *Adjusted R^2* values for your fitted MLR model? Report both values to 4 decimal places.
4. Which single predictor term (feature or dummy-coded category) has the smallest $P > |t|$ value in your model output?
5. Among the numeric predictors (*yr*, *holiday*, *workingday*, *temp*, *atemp*, *hum*, *windspeed*), which one has the largest absolute *t*-value? Provide the feature name and its *t*-value (rounded to 2 decimals).
6. For the categorical variable *weathersit*, which category level decreases predicted *cnt* the most relative to the reference group? Provide the dummy term name and coefficient value.
7. (Standardized coefficients) After standardizing the numeric predictors (but not the dummy codes), which numeric predictor has the largest standardized coefficient magnitude? Provide the feature name and the standardized coefficient value.
8. (Standardized coefficients) Which numeric predictor has the smallest standardized coefficient magnitude (i.e., the weakest effect size on the standardized scale)? Provide the feature name and the standardized coefficient value.
9. Based on your standardized coefficients, name one “best” feature for making informal predictions about daily rentals (large magnitude) and one “worst” feature (small magnitude). Briefly justify your choices in one sentence.



Bike Sharing Practice Answers

These answers were computed by fitting an OLS multiple linear regression model predicting *cnt* using numeric predictors (*yr*, *holiday*, *workingday*, *temp*, *atemp*, *hum*, *windspeed*) plus dummy-coded categorical predictors for *season*, *mnth*, *weekday*, and *weathersit* (with the first category as the reference group for each categorical variable), and excluding leakage variables (*casual* and *registered*).

1. The Bike Sharing daily dataset contains 731 rows and 16 columns.
2. The mean value of *cnt* is 4504.3488.
3. For the fitted MLR model, $R^2 = 0.8381$ and *Adjusted* $R^2 = 0.8312$ (both reported to 4 decimals).
4. The single predictor term with the smallest $P > |t|$ value is *yr* (its p-value is effectively 0.0000 in floating-point terms and is shown as 0.000 in typical summary output due to rounding).
5. Among numeric predictors, *yr* has the largest absolute *t*-value: $t = 34.69$ (rounded to 2 decimals).
6. For *weathersit*, the level that decreases predicted *cnt* the most relative to the reference group is *weathersit_3* with coefficient -2409.68.
7. (Standardized) The numeric predictor with the largest standardized coefficient magnitude is *yr* with standardized coefficient 1738.95.
8. (Standardized) The numeric predictor with the smallest standardized coefficient magnitude is *workingday* with standardized coefficient 17.09.
9. A reasonable “best” feature is *yr* (very large standardized coefficient magnitude), while a reasonable “worst” feature is *workingday* (very

small standardized coefficient magnitude), meaning it contributes little to informal predictions once the other variables are controlled.

More Practice

There are two additional "assignment style" practice assessments below. The first is a repeat of the same dataset you've been using throughout the chapter and will serve as a reminder of everything you have covered. The second uses a new dataset to continue practicing the same concepts.

You may want to test yourself to see how well you understand the concepts. The quiz below walks through the same dataset you just used in these examples above.

This assessment can be taken [online](#).

This assessment can be taken [online](#).

9.11 Assignment

Complete the assignment(s) below (if any):

This assessment can be taken [online](#).